

CASE STUDY

Aplikasi Mobile Sales Force (MSF)

Posisi: Software Engineer

PT Mersifarma

Introduction

A. Peringatan Kerahasiaan

Seluruh informasi, data, dan materi yang diperoleh atau dihasilkan selama pelaksanaan case study ini bersifat rahasia dan hanya digunakan untuk keperluan proses seleksi sebagai Junior Software Engineer. Kandidat dilarang mencatat/memfoto/memindahkan informasi, serta mengungkapkan, mendistribusikan, dan/atau menggunakan sebagian maupun seluruh informasi tersebut kepada pihak lain dalam kondisi apapun.

B. Objektif

Membangun Backend API dan Frontend Web App untuk aplikasi Mobile Sales Force (MSF) yang mencakup tiga fitur utama — **Call List, Call Plan, dan Call Actual** — dengan tetap mematuhi business rules, approval workflow, dan validasi data yang berlaku.

C. Tujuan Case Study

Membangun Backend API dan Frontend Web App aplikasi MSF. Dalam case study ini, kandidat berperan sebagai Junior Software Engineer dengan kondisi berikut:

- Bebas menentukan bahasa pemrograman, framework, dan database sesuai penguasaan masing-masing.
- Bertanggung jawab atas perancangan, implementasi, dan pengujian fitur yang dikerjakan.

Catatan: Kandidat tidak diwajibkan menghasilkan aplikasi berskala produksi. Fokus utama yang dievaluasi adalah cara berpikir, kualitas kode, pemahaman business rules, dan kemampuan mendemonstrasikan hasil kerja. Jika dirasa ada data yang kurang, kandidat bisa membuat asumsi sendiri, harap asumsi disampaikan atau dituliskan dengan jelas di README.

D. Ruang Lingkup & Kebebasan Pendekatan

Kandidat dibebaskan untuk menentukan metode dan solusi terbaik untuk menyelesaikan case study ini. Aspek yang dapat kandidat tentukan sendiri meliputi:

- Bahasa pemrograman & framework backend (Node.js, Python, PHP, Go, Java, Ruby, atau lainnya)
- Database yang digunakan (PostgreSQL, MySQL, SQLite, MongoDB, dsb)
- Framework frontend (React, Vue, Next.js, Nuxt, Svelte, Angular, atau vanilla HTML+JS)
- Mekanisme autentikasi (hardcoded token, session, atau JWT)
- Struktur repository (folder backend/ dan frontend/ terpisah atau monorepo)

Aspek Strategis

- Kesesuaian rancangan API dan struktur data dengan business rules dan tujuan bisnis.
- Pemahaman user journey dari **Call List → Call Plan → Call Actual**.
- Pemahaman trade-off antara kecepatan pengerjaan, kelengkapan fitur, dan kualitas kode.

Aspek Eksekusi

- Kelengkapan implementasi endpoint dan halaman sesuai deliverable.
- Kualitas kode: struktur, keterbacaan, penanganan error, dan validasi input.
- Kemudahan menjalankan project secara lokal berdasarkan dokumentasi README.

Aspek Compliance

- Sensitivitas terhadap business rules dan approval workflow.
- Kemampuan mendeteksi edge cases dan skenario negatif.
- Konsistensi validasi antara sisi backend dan frontend.

E. Kriteria Penilaian

- Struktur berpikir & problem solving.
- Keseimbangan antara kelengkapan fitur, kualitas kode, dan kepatuhan business rules.
- Kualitas komunikasi & justifikasi keputusan teknis.
- Pemanfaatan tools secara kritis.

F. Panduan Standar Pengerjaan

1. Fase Discovery & Persiapan

Tujuan: Memahami aplikasi, user, dan scope fitur.

- Identifikasi target user (MR, DM, RSM, MM) dan alur kerja masing-masing
- Analisis singkat fitur utama dan business rules
- Penyiapan repository, struktur folder, dan environment lokal

2. Fase Perancangan Data & Skema

Tujuan: Menyiapkan fondasi data yang konsisten.

- Perancangan skema tabel untuk users, roles, master customer list (MCL), dan products
- Penyusunan SEED.sql atau migration untuk data master

- Penentuan relasi antar entitas Call List, Call Plan, dan Call Actual

3. Fase Backend Development

Tujuan: Menyediakan API yang stabil dan tervalidasi.

- Implementasi endpoint Call List, Call Plan, dan Call Actual
- Implementasi approval workflow beserta status flow-nya
- Validasi input dan error handling yang konsisten

4. Fase Frontend Development

Tujuan: Menyediakan ui yang fungsional dan rapi.

- Implementasi halaman Call List, Call Plan, dan Call Actual
- Integrasi ke API backend beserta loading state dan error handling
- Tampilan responsive karena MR mengakses aplikasi melalui perangkat mobile

5. Fase Validasi Business Rules & Testing

Tujuan: Memastikan aturan bisnis benar-benar berjalan.

- Penyusunan skenario test manual untuk happy path dan error case
- Pengujian edge case seperti double visit dan approval oleh role yang bukan atasan langsung
- Dokumentasi hasil pengujian dalam bentuk screenshot atau screen recording

6. Fase Pemanfaatan Tools

Tujuan: Menunjukkan workflow dan pemanfaatan tools.

- Tools yang digunakan & untuk apa
- Contoh output tools yang diterima vs ditolak (beserta alasan)
- Pendekatan validasi output tools sebelum dipakai di dalam kode

7. Fase Dokumentasi & Submission

Tujuan: Memastikan hasil kerja mudah dijalankan dan dinilai.

- README.md berisi cara install, cara run, dan tech stack yang dipakai
- Dokumentasi skenario test dan refleksi pengerjaan
- Penyerahan link repository dan link deploy (jika ada) ke interviewer

Case Study

Latar Belakang Case Study

PT Mersifarma, perusahaan farmasi nasional, sedang membangun Aplikasi Mobile Sales Force (MSF) untuk para Medical Representative (MR) / Field Force (FF) guna mengelola kunjungan ke dokter secara digital. Kandidat diminta membangun Backend API dan Frontend Web App untuk tiga fitur utama: Call List, Call Plan, dan Call Actual.

Profil Aplikasi:

- Nama Aplikasi: Mobile Sales Force (MSF)
- Lingkup Pengerjaan: Backend API + Frontend Web App
- Kategori: Aplikasi manajemen kunjungan dokter oleh MR
- User Utama: Medical Representative (MR) / Field Force (FF)
- User Sekunder: District Manager (DM), Regional Sales Manager (RSM), Marketing Manager (MM)
- Autentikasi: Sederhana (hardcoded token / session / JWT)
- Deployment: Opsional

Fitur Utama:

1. Call List Bulanan — MR mengajukan daftar dokter target kunjungan untuk 1 bulan ke depan.
2. Approval Workflow — pengajuan Call List cukup di-approve oleh atasan langsung satu level di atas pengaju (MR di-approve DM, DM di-approve RSM, RSM di-approve MM). Call List valid jika sudah approved.
3. Call Plan — MR membuat rencana kunjungan harian berdasarkan Call List yang approved.
4. Call Actual — Aktualisasi kunjungan: check-in/check-out, upload foto, tanda tangan (TTD), dan detailing produk.
5. Unplan Visit — Kunjungan tanpa Call Plan ke dokter yang sudah ada di Call List approved.
6. Non Target Visit — Kunjungan ke dokter yang ada di Master Customer List (MCL), meski tidak masuk Call List.

Business Rules Penting:

- Status flow Call List: draft → submitted → pending_approval → approved / rejected.
- Approval bersifat single-level: cukup di-approve oleh atasan langsung satu level di atas pengaju — MR oleh DM, DM oleh RSM, RSM oleh MM.
- Jika direject, Call List kembali ke status draft dan pengaju harus merevisi lalu submit ulang ke atasan langsung yang sama.
- Call Plan hanya bisa dibuat dari Call List yang berstatus approved.
- Satu dokter hanya boleh memiliki satu Call Plan per hari.
- Tanggal Call Plan harus berada dalam periode bulan Call List.
- Maksimal 1 visit per dokter per hari, dicek terhadap seluruh data Call Actual.
- photo_url dan signature_url wajib diisi pada setiap Call Actual.

- Detailing produk minimal 1 item pada setiap Call Actual.
- Jika plan_id diisi, doctor_id harus sesuai dengan Call Plan terkait.
- Jika plan_id null, doctor_id harus terdaftar di MCL (Non Target Visit).

Kondisi saat ini:

- MR akan mengakses aplikasi di lapangan melalui perangkat mobile, sehingga tampilan harus responsive.
- Data kunjungan sangat krusial untuk perhitungan incentive dan reporting ke manajemen.

Tujuan Pengembangan

Tim Engineering ingin membangun fondasi aplikasi yang:

- Memastikan fitur Call List, Call Plan, dan Call Actual berjalan sesuai business rules.
- Memastikan approval workflow single-level berjalan benar di setiap jenjang (MR → DM, DM → RSM, RSM → MM).
- Memastikan validasi kunjungan mencegah terjadinya double visit pada dokter yang sama.
- Memastikan data kunjungan lengkap: foto, tanda tangan, dan detailing produk.
- Menyediakan antarmuka web yang fungsional dan nyaman diakses dari perangkat mobile.
- Menghasilkan struktur kode dan dokumentasi yang bisa dikembangkan jangka panjang.

Di sisi lain, solusi ini harus:

- Mudah dijalankan secara lokal oleh reviewer tanpa konfigurasi rumit.
- Mampu menangani edge cases yang umum terjadi di lapangan.

Sumber Daya yang Tersedia

Sumber Daya	Detail
Tools	Bebas memilih bahasa pemrograman, framework, database, dan tools pendukung sesuai kebutuhan
Aset Awal	Spesifikasi fitur, business rules, daftar endpoint yang diminta, dan struktur data master (MCL, Products, Users)
Environment	Environment lokal; deployment ke cloud bersifat opsional (nice to have)

Deliverable yang Diminta

Kandidat diminta membangun Backend API dan Frontend Web App untuk aplikasi MSF, dengan output berupa repository yang dapat dijalankan beserta dokumentasinya. Deliverable terdiri dari 4 bagian dengan bobot penilaian sebagai berikut:

Bagian 1 — Backend API (Bobot 40%)

Implementasikan endpoint berikut beserta validasi dan error handling. Sertakan pula SEED.sql atau migration untuk data master: users (id, name, email, role, region_id), roles (mr, dm, rsm, mm), master_customers / mcl (id, name, specialization, address, phone), dan products (id, name, category).

Call List

Method	Endpoint	Deskripsi
POST	/api/call-lists	MR membuat Call List bulanan (pilih bulan + array doctor_ids dari MCL)
GET	/api/call-lists/:id	Detail Call List beserta status approval
PATCH	/api/call-lists/:id/approve	Approve / reject oleh atasan langsung pengaju (body: {status, reason})
GET	/api/call-lists	List Call List by MR yang login

Rules: Status flow draft → submitted → pending_approval → approved / rejected; approval bersifat single-level oleh atasan langsung satu level di atas pengaju (MR oleh DM, DM oleh RSM, RSM oleh MM); jika direject, Call List kembali ke draft dan pengaju dapat mengedit serta submit ulang.

Call Plan

Method	Endpoint	Deskripsi
POST	/api/call-plans	MR membuat Call Plan dari Call List yang approved
GET	/api/call-plans	List Call Plan by MR

Rules: Call Plan hanya bisa dibuat dari Call List berstatus approved; satu dokter hanya boleh memiliki satu plan per hari; tanggal plan harus berada dalam periode bulan Call List.

Call Actual (Visit)

Method	Endpoint	Deskripsi
POST	/api/call-actuals	MR submit hasil kunjungan
GET	/api/call-actuals	List Call Actual by MR

Contoh body request:

```
{
  "plan_id": 1,
  "doctor_id": 5,
```

```
"visit_date": "2026-08-12",
"check_in": "08:30:00",
"check_out": "09:00:00",
"photo_url": "https://...",
"signature_url": "https://...",
"products": [1, 3, 5]
}
```

Keterangan: plan_id bersifat nullable (null berarti Unplan / Non Target Visit), sedangkan products berisi array product_id yang didetailkan.

Rules: photo_url dan signature_url wajib ada; products minimal 1 item; maksimal 1 visit per dokter per hari dengan pengecekan ke seluruh data call_actuals; jika plan_id diisi maka doctor_id harus cocok dengan plan; jika plan_id null maka doctor_id harus terdaftar di MCL (Non Target Visit).

Bagian 2 — Frontend Web App (Bobot 40%)

Bangun 3 halaman utama. Desain tidak perlu kompleks, cukup fungsional dan rapi.

- Halaman Call List: form pemilihan bulan dan multi-select dokter dari MCL, tabel daftar Call List beserta status approval, serta tombol submit untuk mengubah status dari draft menjadi submitted.
- Halaman Call Plan: form pemilihan Call List approved, dokter, tanggal, dan jam; tabel daftar Call Plan; serta validasi error apabila dokter sudah memiliki plan di tanggal yang sama.
- Halaman Call Actual: form pemilihan Call Plan (atau mode Unplan / Non Target), input check-in dan check-out, upload foto (mock URL), input tanda tangan (mock URL), multi-select produk, tabel daftar Call Actual, serta status In Progress → Completed.
- Requirements Frontend: integrasi ke API backend (fetch/axios), loading state dan error handling (toast/alert), serta tampilan responsive karena MR akan mengakses melalui perangkat mobile.

Bagian 3 — Business Rules Validation & Testing (Bobot 15%)

Di dalam README.md, sertakan minimal 5 skenario test manual yang mencakup happy path dan error case:

- MR membuat Call List → submit → DM (atasan langsung) approve → MR membuat Call Plan → eksekusi Call Actual.
- MR mencoba membuat Call Plan dari Call List yang belum approved → expected error.
- MR mencoba melakukan visit ke dokter yang sama dua kali dalam satu hari → expected error.
- MR submit Call Actual tanpa foto → expected error.
- Call List milik MR dicoba di-approve oleh role yang bukan atasan langsungnya (misalnya RSM atau MM) → expected error.

Bagian 4 — Refleksi & Tools (Bobot 5%)

Di dalam README.md, tuliskan secara singkat:

- Arsitektur berserta cara menjalankan aplikasi

Alur Pengerjaan & Penyajian

- Discovery & Persiapan — memahami fitur, user, business rules, dan menyiapkan repository.
- Perancangan Data — menyusun skema tabel dan seed data master.
- Eksekusi Deliverable — mengerjakan Backend API dan Frontend Web App.
- Presentasi & Q&A — 30.

Tantangan yang Harus Dipertimbangkan

Dalam merancang solusi, kandidat diharapkan mempertimbangkan tantangan berikut:

- Bagaimana jika MR melakukan visit ke dokter yang sama di hari yang sama melalui berbagai jalur (Planned + Unplan + Non Target)? Bagaimana sistem mencegah double visit?
- Bagaimana jika approval workflow terhenti di satu level (misalnya DM tidak approve selama sehari-hari)? Apa mekanisme reminder atau escalation-nya?
- Bagaimana jika Call List direject setelah Call Plan terlanjur dibuat? Bagaimana perlakuan terhadap data turunannya?
- Bagaimana jika MR submit Call Actual dengan tanggal di luar periode Call List? Validasi apa yang dijalankan?
- Bagaimana memastikan validasi business rules konsisten antara sisi backend dan frontend?
- Bagaimana skala solusi ini jika setelah 3 bulan harus menambah fitur baru (misalnya offline mode atau GPS tracking)?

Future Development (Bonus)

Untuk kandidat yang ingin menunjukkan kapasitas lebih, boleh mencakup pengembangan berikut:

- Implementasi Unplan Visit (Call Actual tanpa Call Plan, dokter harus ada di Call List approved).
- Implementasi Non Target Visit (Call Actual tanpa Call Plan, dokter bebas dari MCL).
- Offline mode sederhana (menyimpan form di localStorage dan sync saat online).
- Unit test minimal 3 test untuk 1 fungsi kritis.
- Pagination dan filter pada list Call Plan / Call Actual.
- Authentication sederhana (login by role, token JWT).
- Deploy ke cloud beserta link live.

Aturan Main

Aturan	Ketentuan
Durasi pengerjaan	5 hari kerja sejak case dikirim.

Aturan	Ketentuan
Format output	Push ke repository GitHub (public), lalu kirim link repo.
Struktur repo	Folder backend/ dan frontend/ (atau monorepo), disertai README.md dan SEED.sql / migration.
Tech stack	Bahasa pemrograman, framework, dan database bebas — pilih yang paling dikuasai.
Deployment	Opsional (nice to have).
Sesi presentasi	30 menit
Asumsi	Kandidat bebas membuat asumsi tambahan, asalkan dituliskan dengan jelas di README.

Catatan Akhir: Case study ini dirancang untuk mengukur cara berpikir dan pendekatan teknis kandidat — bukan kesempurnaan output. Yang dinilai adalah cara berpikir, kualitas kode, validasi business rules, dan kemampuan mendemokan hasil kerja. Kami menghargai kandidat yang berani menunjukkan proses berpikir dan keterbatasannya dengan transparan.

— Selamat mengerjakan —